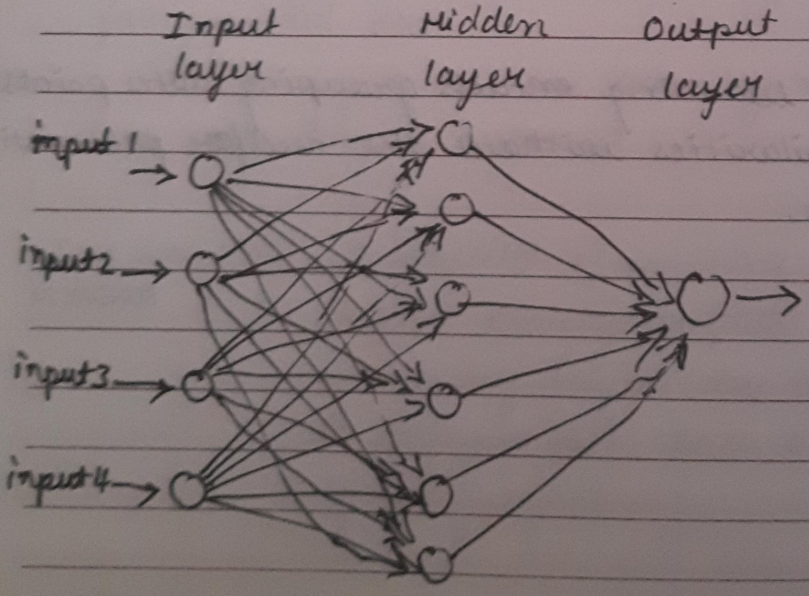
Neural Networks

Neural networks are set of algorithms inspired by functioning of human brain. Neurons are data processing cells in our brain.

Similarly artificial neural networks have artificial neurons. They take large sets of data as i/p and process the data i.e. draws out patterns in the data.

ANNs (Artificial neural networks) are composed of a large number of highly interconnected elements i.e. neurones working in unision to solve a specific problem like data classification, Image recognition, voice recognition through a learning process

Network layers



input layer - Raw information fed into the network

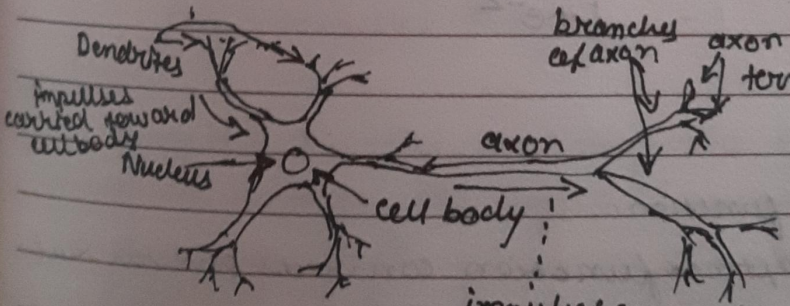
Hidden layer - Activity of each hidden units is determined by activities of ~~each~~ input units and weights on the connection between input and ~~output~~ hidden units

output layer - The layer of output units

The behaviour of output units depend on activity of hidden units and weights between hidden and output units

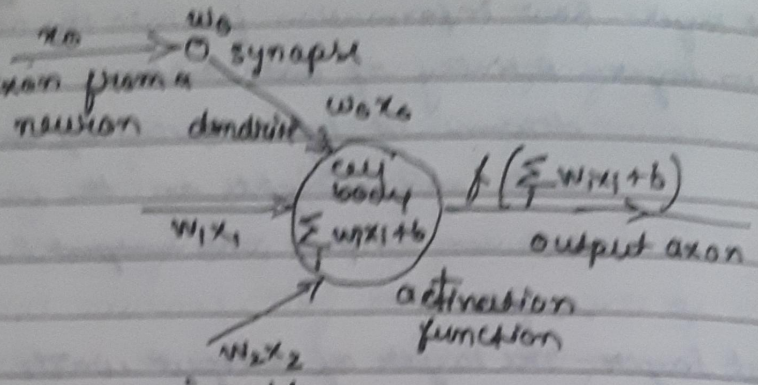
A single neuron

- Also called as a node or unit.
- It receives input from other neurons (nodes) or from an external source and computes an output
- Each input has an associated weight which is assigned on basis of relative importance to other inputs



Biological neuron.

27 Artificial neuron. (mathematical model)



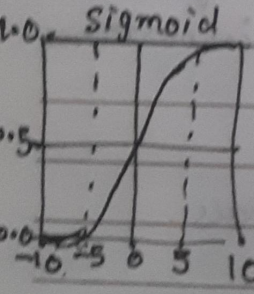
takes weighted $x_1, x_2, \ldots x_i$ with weights $w_1, w_2, \ldots w_i$ and bias b

The function $\sum_i w_i x_i + b$ is called an activation function. The purpose of this function is to introduce non-linearity to neuron. This enables neuron to learn from non-linear data in sudoku rgb

Every activation function performs certain mathematical operations on it

There are several activation functions

- Sigmoid - Takes real valued input and converts it to range between 0 and



$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Softmax function -

The softmax function converts a vector real number (logits) into probabilities.

These probabilities range betn 0 and 1

Sum up to 1, making them interpretable as probabilities of different outcomes

how does it work?

1. Inputs (logits)

suppose you have a vector of raw scores or logits from a model of 3 classes, e.g., $[z_1, z_2, z_3]$

2. Exponentiation:

For each score z_i, calculate e^{z_i}. This step ensures all values are positive

3. Normalization:

Divide e^{z_i} by the sum of exponentials of all logits

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

p_i is the softmax probability for class i.

The denominator $\sum_j e^{z_j}$ normalizes the probabilities to sum up to 1

Probability (pass) + Probability (fail) = 1

tanh: takes a real valued input and squashes it to range $[-1, 1]$

$$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$$

Imagine a model predicting the type of pet (dog, cat, rabbit). It outputs the logits $[2.0, 1.0, 0.1]$

1. Exponentiate e^z

$[e^{2.0}, e^{1.0}, e^{0.1}] = [7.39, 2.72, 1.11]$

2. Normalize

$p_{dog} = 7.39 / [7.39 + 2.72 + 1.11] = 0.65$

$P_{cat} = \frac{2.72}{7.39 + 2.72 + 1.11} \approx 0.24$

$P_{rabbit} = \frac{1.11}{7.39 + 2.72 + 1.11} \approx 0.10$

Final probabilities = [0.65, 0.24, 0.10]

Interpretation

The model assigns a 65% probability to dog, 24% to cat, 10% to rabit

Why softmax ?

The output probabilities help differentiate between class or types, hence makes it suitable for gradient based optimisation. (e.g. back propagation)

The probabilities show the models confidence for each class.

Softmax pairs naturally with cross-entropy loss function, which measures how well predicted probabilities match the true class labels.

Practical notes:

1. Numerical Stability

The exponentials in e^{z_i} can lead to overflow for large values of z. To stabilize

Substract the maximum logit value from all logits

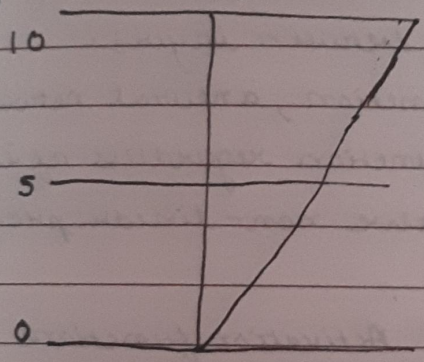
$z_i = z_i - \max(z)$

2. Use case

The softmax function is typically used as activation function in final layer of neural networks designed for classification tasks with multiple classes

ReLU - Stands for Rectified linear Unit. It takes real valued input and thresholds it at 0. i.e. replaces negative values with zero.

$f(x) = \max(0, x)$



Bias - The main function of Bias is to provide every node with trainable constant value.

Bias value allows you to shift the activation function to left or right.

The output of network is computed by multiplying the input (x) by weight (W_0) and passing result through an activation function like (Sigmoid)

changing weight changes steepness of sigmoid.

But what if want the network output to 0 when x is 2. This is where bias comes in. Just changing steepness won't work, we want entire curve to be shifted to right. Bias allows us to do that.

SUNDAY 31

01 ~~Step~~ - Activation function

Activation functions are mathematical functions applied to the output of neuron in a neural network.

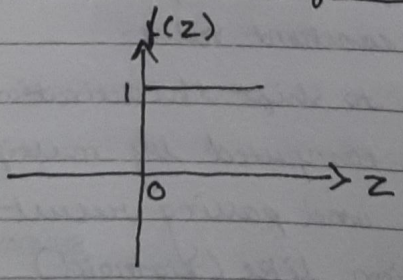
It determines whether the neuron should be activated (pass information forward) and introduces non-linearity, allowing neural networks to model complex patterns and relationships

It enables neural network to learn complex relations /non-linear relationships between layers.

Without activation function, a neural network would behave like a linear function regardless of its depth, limiting its ability to solve non-linear problems.

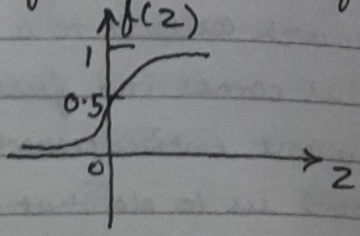
Examples of
~~Key Concepts behind~~ Activation functions

Step Activation function



if z is less than or equal to 0, output is 0.
if z is greater than 0, output is 1

Sigmoid Activation function



$$f(z) = \frac{1}{1+e^{-z}}$$

if $z \geq 0.5$, output is 1
$z < 0.5$, output is 0

threshold = 0.5